

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS

MANUAL TÉCNICO

Integrantes:

Anderson Mejía 202308347
Byron Antonio Choy 202100285

Curso: Practicas Iniciales Sección C
Guatemala, 10 septiembre de 2026

Introducción

Este documento describe la arquitectura técnica, el modelo de datos y la especificación de la API REST del Sistema de Calificación de Cursos, desarrollado como proyecto del curso de la Escuela de Ciencias y Sistemas de la Facultad de Ingeniería, Universidad de San Carlos de Guatemala.

El sistema permite a los estudiantes de Ciencias y Sistemas publicar, evaluar y comentar sobre catedráticos y cursos, así como gestionar su perfil académico y su historial de cursos aprobados. El presente manual está dirigido al equipo docente evaluador y a cualquier desarrollador que necesite dar mantenimiento o continuar el desarrollo del sistema.

Objetivo del documento

- Describir la arquitectura Cliente-Servidor implementada y las tecnologías utilizadas.
- Documentar el modelo de datos relacional, incluyendo el diagrama Entidad-Relación y el diccionario de datos.
- Especificar cada endpoint de la API REST: método HTTP, ruta, autenticación requerida y estructura de datos esperada.

Estructura de carpetas del backend

```
backend/
├─ package.json
├─ .env          (no versionado – ver sección 6.1)
├─ .gitignore
└─ src/
    ├─ index.js    (arranque del servidor y montaje de rutas)
    ├─ config/
    │   └─ db.js    (pool de conexiones MySQL)
    ├─ middlewares/
    │   └─ auth.js   (verificarToken – validación de JWT)
    └─ routes/
        ├─ auth.routes.js
        ├─ users.routes.js
        ├─ posts.routes.js
        ├─ comments.routes.js
        ├─ courses.routes.js
        └─ professors.routes.js
```

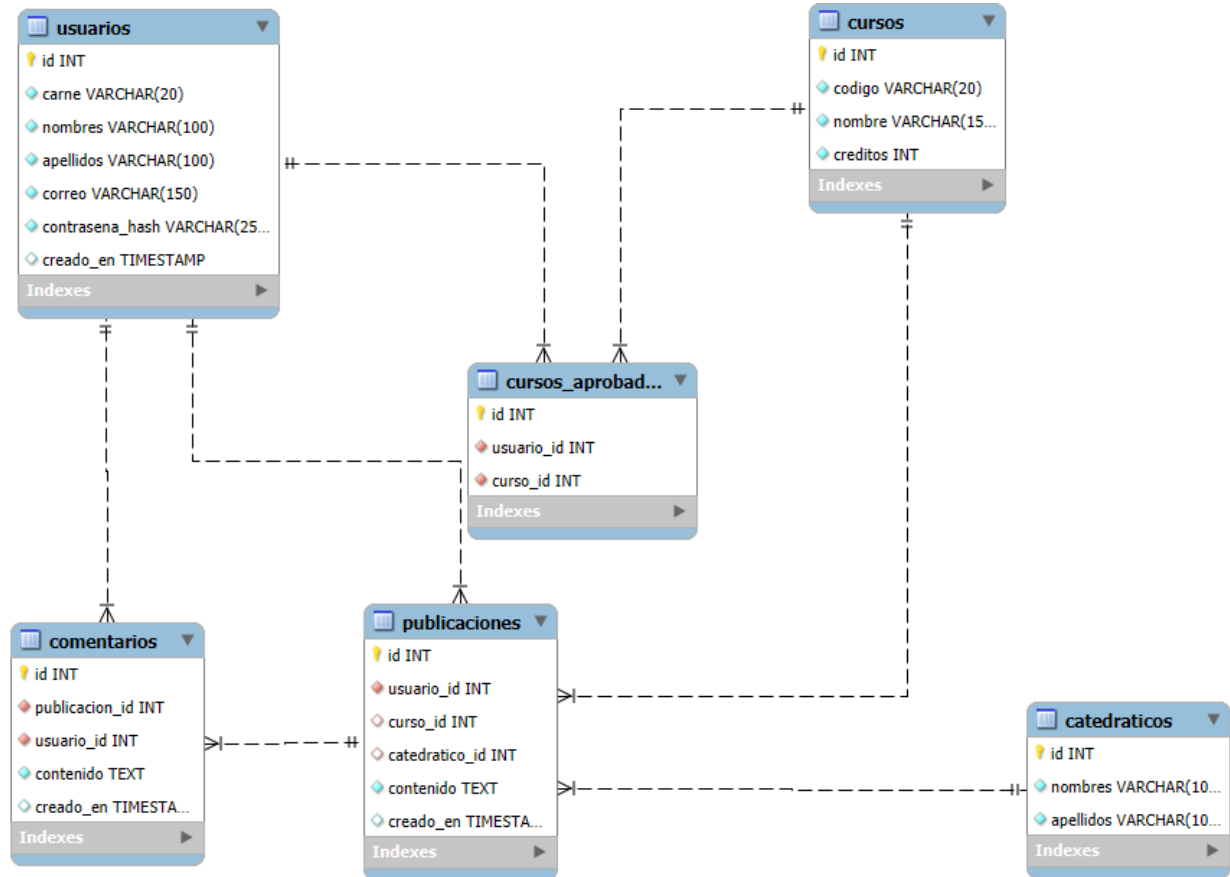
Flujo general de una petición autenticada

- El cliente envía la petición HTTP incluyendo el header Authorization: Bearer <token>.
- El middleware verificarToken (src/middlewares/auth.js) valida el token con jwt.verify usando JWT_SECRET.
- Si el token es válido, se adjunta el payload decodificado a req.usuario y la petición continúa al controlador de la ruta.
- El controlador ejecuta la consulta correspondiente contra MySQL mediante el pool exportado por config/db.js.
- Si el token falta o es inválido, el middleware responde 401 o 403 sin llegar a ejecutar la lógica de negocio.

Estructura de carpetas del frontend

```
frontend/
├─ index.html
├─ package.json
├─ vite.config.js
├─ .env                (VITE_API_URL – no versionado)
├─ .gitignore
├─ src/
│   ├─ main.jsx        (punto de entrada; envuelve <App/> en AuthProvider)
│   ├─ App.jsx         (BrowserRouter + Navbar + AppRoutes)
│   ├─ index.css        (estilos globales, variables CSS)
│   ├─ context/
│   │   └─ AuthContext.jsx (usuario, login, logout – persistidos en localStorage)
│   ├─ routes/
│   │   ├─ AppRoutes.jsx (definición de rutas)
│   │   └─ RutaProtegida.jsx (guard: redirige a /login si no hay sesión)
│   ├─ services/
│   │   ├─ api.js       (instancia de axios; inyecta el JWT en cada petición)
│   │   └─ catalogos.service.js (obtenerCursos, obtenerCatedraticos)
│   ├─ components/
│   │   ├─ Navbar.jsx
│   │   └─ PublicacionCard.jsx
│   └─ pages/
│       ├─ Login/Login.jsx
│       ├─ Registro/Registro.jsx
│       ├─ RecuperarContrasena/RecuperarContrasena.jsx
│       ├─ Home/Home.jsx
│       ├─ CrearPublicacion/CrearPublicacion.jsx
│       ├─ BuscarPerfil/BuscarPerfil.jsx
│       └─ Perfil/Perfil.jsx
```

Modelo de Datos



Diccionario de datos

usuarios

Almacena las cuentas de los estudiantes registrados en la plataforma.

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental del usuario
carne	VARCHAR(20)	UNIQUE	Registro académico del estudiante; identificador público
nombres	VARCHAR(100)		Nombres del usuario
apellidos	VARCHAR(100)		Apellidos del usuario
correo	VARCHAR(150)	UNIQUE	Correo electrónico institucional

contrasena_hash	VARCHAR(255)		Hash bcrypt de la contraseña; nunca se almacena en texto plano
creado_en	TIMESTAMP		Fecha y hora de creación del registro

cursos

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental del curso
codigo	VARCHAR(20)	UNIQUE	Código oficial del curso según el pensum
nombre	VARCHAR(150)		Nombre del curso
creditos	INT		Créditos académicos que otorga el curso

catedráticos

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental del catedrático
nombres	VARCHAR(100)		Nombres del catedrático
apellidos	VARCHAR(100)		Apellidos del catedrático

publicaciones

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental de la publicación
usuario_id	INT	FK → usuarios.id	Autor de la publicación
curso_id	INT	FK → cursos.id (NULL)	Curso evaluado; excluyente con catedratico_id

catedratico_id	INT	FK → catedraticos.id (NULL)	Catedrático evaluado; excluyente con curso_id
contenido	TEXT		Texto de la opinión
creado_en	TIMESTAMP		Fecha y hora de creación

comentarios

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental del comentario
publicacion_id	INT	FK → publicaciones.id	Publicación a la que pertenece el comentario
usuario_id	INT	FK → usuarios.id	Autor del comentario
contenido	TEXT		Texto del comentario
creado_en	TIMESTAMP		Fecha y hora de creación

cursos aprobados

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador autoincremental del registro
usuario_id	INT	FK → usuarios.id	Estudiante al que pertenece el curso aprobado
curso_id	INT	FK → cursos.id	Curso aprobado

Especificación de Endpoints — API REST

Todas las rutas están montadas bajo el prefijo /api. Las rutas marcadas con Auth: JWT requieren el header Authorization: Bearer <token> obtenido en el login; su ausencia o invalidez responde 401/403 respectivamente.

4.1 Autenticación — /api/auth

Método	Ruta	Auth	Descripción / Body esperado
POST	/api/auth/register	No	Body: { carne, nombres, apellidos, correo, password }. Encripta la contraseña con bcrypt e inserta el usuario. 400 si falta un campo o si el carné/correo ya existe.
POST	/api/auth/login	No	Body: { carne, password }. Verifica credenciales y responde { token, usuario }. El token expira en 8 horas.
POST	/api/auth/recover	No	Body: { carne, correo, nuevaPassword }. Si carné y correo coinciden, actualiza el hash de la contraseña.

4.2 Usuarios y perfil — /api/users

Método	Ruta	Auth	Descripción / Body esperado
GET	/api/users/:carne	No	Devuelve los datos públicos del perfil (sin contraseña) buscando por carné.
PUT	/api/users/:carne	JWT	Body: { nombres, apellidos, correo }. Solo el dueño del perfil (carné del token) puede editarlo; responde 403 en caso contrario.
GET	/api/users/:carne/courses	No	Devuelve { cursos, totalCreditos } con el listado de cursos aprobados y la suma de créditos.
POST	/api/users/:carne/courses	JWT	Body: { curso_id }. Agrega un curso aprobado al expediente del usuario autenticado. 400 si ya estaba registrado.
DELETE	/api/users/:carne/courses/:curso_id	JWT	Elimina un curso aprobado del expediente del usuario autenticado. 404 si no existía.

Publicaciones— /api/posts

Método	Ruta	Auth	Descripción / Body esperado
GET	/api/posts	No	Query params opcionales y combinables: curso_id, catedratico_id, busqueda (texto libre sobre contenido, curso o catedrático). Resultado ordenado por creado_en descendente.
POST	/api/posts	JWT	Body: { curso_id XOR catedratico_id, contenido }. Debe indicarse exactamente una entidad evaluada; 400 si faltan ambas o se envían las dos.
GET	/api/posts/auxiliares/datos	No	Devuelve { cursos, catedraticos } para poblar los selectores del formulario de publicación.

4.4 Comentarios — /api/comments

Método	Ruta	Auth	Descripción / Body esperado
GET	/api/comments/:publicacion_id	No	Devuelve los comentarios de una publicación en orden cronológico ascendente, con datos del autor.
POST	/api/comments/:publicacion_id	JWT	Body: { contenido }. Agrega un comentario asociado al usuario autenticado. 400 si el contenido viene vacío.

Catálogos — /api/courses y /api/professors

Método	Ruta	Auth	Descripción / Body esperado
GET	/api/courses	No	Devuelve el listado completo de cursos (id, codigo, nombre, credits) ordenado alfabéticamente por nombre.
GET	/api/professors	No	Devuelve el listado completo de catedráticos (id, nombres, apellidos) ordenado alfabéticamente por apellido.

4.6 Utilitario

Método	Ruta	Auth	Descripción / Body esperado
GET	/api/health	No	Ruta de verificación básica de disponibilidad del servidor; responde { status: 'ok' }.

Configuración y Despliegue

Variables de entorno — Backend (.env)

El archivo .env debe crearse en la raíz del backend y nunca debe subirse al repositorio (debe listarse en .gitignore). Contiene las siguientes variables:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=<contraseña de MySQL>
DB_NAME=calificacion_cursos
JWT_SECRET=<cadena secreta larga y aleatoria>
PORT=5000
```

Variables de entorno — Frontend (.env)

El archivo .env del frontend es independiente del backend y también debe excluirse del repositorio. Vite únicamente expone al cliente las variables que comienzan con el prefijo VITE_:

```
VITE_API_URL=http://localhost:5000/api
```

Pasos para levantar el entorno local

- 1. Backend: instalar dependencias con `npm install` dentro de la carpeta backend.
- 2. Crear la base de datos ejecutando `schema.sql`: `mysql -u root -p < schema.sql`
- 3. Poblar las tablas `courses` y `teachers` con la oferta académica vigente (ver sección 6.4).
- 4. Crear el archivo `.env` del backend con las variables de la sección 6.1.
- 5. Iniciar el backend en modo desarrollo: `npm run dev` (usa nodemon sobre `src/index.js`), puerto 5000 por defecto.
- 6. Verificar en consola la conexión exitosa a MySQL y el mensaje de servidor escuchando.
- 7. Frontend: instalar dependencias con `npm install` dentro de la carpeta frontend.
- 8. Crear el archivo `.env` del frontend con la variable de la sección 6.2, apuntando al backend ya levantado.
- 9. Iniciar el cliente con `npm run dev` (Vite) y abrir la URL local que indica la consola (por defecto `http://localhost:5173`).